NL2Semantic2SQL: Semantic Grounding and Safe Execution for PostGIS-based NL2GeoSQL, with a Cross-Domain Stress Test on Warehouse Queries

Anonymous Author(s)
Affiliation withheld for double-anonymised review

Abstract

Natural language interfaces to geospatial databases must handle challenges that mainstream text-to-SQL benchmarks rarely exercise: geometry-aware schema grounding, OGC/PostGIS topological and metric predicates, coordinate reference systems, `::geography` casting, KNN operators, and safety-critical refusal of destructive or unbounded queries. We present **NL2Semantic2SQL**, a semantic-grounding and safe-execution framework for executable PostGIS NL2GeoSQL. It couples a semantic layer (aliases, geometry types, SRIDs, spatial operator rules) with an intent-conditioned router, schema-aware prompting, few-shot retrieval, a SQL postprocessor that enforces read-only safety and injects `LIMIT` on large-table preview queries, and an execution-time self-correction loop. The same bilingual architecture serves Chinese GIS and English warehouse queries from a single pipeline; we evaluate both sides so that the current state of each is measurable. We evaluate on a 125-question Chongqing PostGIS benchmark (85 Spatial + 40 Robustness across six safety categories) and on a warehouse evaluation track on BIRD mini_dev, reported as a 108-question design set and a disjoint 150-question held-out set. On GIS Spatial, the full pipeline reaches EX= 0.682 vs. baseline 0.529 (paired McNemar two-sided exact $p = 0.0072$; $+0.118$ over DIN-SQL with $p = 0.0213$). The 40-question Robustness suite yields safe-refusal 1.000 and bounded-answer compliance $33/40 = 0.825$, with one engineering gap (OOM Prevention bounded-answer $1/8$) we explain and describe a fix for. On BIRD, the pipeline improves the design set by $+0.093$ ($p = 0.0213$) but only $+0.033$ on the held-out set ($p = 0.3833$, 95% CI $[-0.03, 0.09]$); this current-state snapshot motivates a planned second round of warehouse-side grounding. A 50-question Chinese BIRD probe using LLM-translated questions is reported as exploratory.

Keywords: NL2GeoSQL; PostGIS; natural language interface; geospatial database; semantic layer; spatial predicates; coordinate reference systems; safe execution; large language model.

1 Introduction

Natural language access to spatial databases is a long-standing GIScience problem [1, 2, 3]. Recent large language models have pushed general text-to-SQL forward on benchmarks such as Spider [15] and BIRD [20], yet these benchmarks deliberately abstract away the execution machinery that defines geospatial querying: the OGC Simple Feature Access predicates [4], coordinate reference system transformations, the `geometry`/`geography` type distinction, spatial indexes, KNN operators, and the safety constraints that any production GIS must enforce against destructive or unbounded queries. Natural language to executable PostGIS [6] (NL2GeoSQL) is therefore not an instance of text-to-SQL with a GIS dataset but a distinct problem shaped by spatial-database semantics.

The grounding stage is where the gap is most visible. General text-to-SQL schema linking aligns user expressions with column names; PostGIS grounding must additionally recognise geometry-bearing attributes, infer whether a question requires a topological predicate (`ST_Intersects`, `ST_Touches`,

ST_Contains), a metric operator (ST_Distance, ST_DWithin, ST_Area), a buffer or KNN operator (<->), and decide when a geography cast or ST_Transform is required so that area and distance are returned in metres rather than degrees. It must also refuse destructive intent (DELETE, UPDATE, DROP) and enforce bounded output on large-table preview queries. These are GIScience computations, not prompt-engineering problems.

We present **NL2Semantic2SQL**, a semantic-grounding and safe-execution framework for PostGIS-based NL2GeoSQL. A semantic layer provides aliases, geometry metadata, spatial operator rules, and safety constraints; a bilingual intent classifier routes Chinese GIS and English warehouse queries into the appropriate grounding path; a SQL postprocessor enforces read-only safety, repairs identifier quoting, and injects LIMIT on large-table preview queries; and an execution-time self-correction loop recovers from common PostGIS execution errors such as missing geography casts and mixed-case identifier quoting. The same framework serves both GIS and warehouse users, and we evaluate both sides so that the current state of each is measurable.

For primary evaluation, we construct a 125-question Chongqing PostGIS benchmark with executable gold SQL (85 Spatial covering OGC-Simple-Feature predicates, metric and KNN operators, CRS transforms, and aggregation across three difficulty levels; plus 40 Robustness spanning six safety and refusal categories). For the warehouse evaluation track, we use BIRD mini_dev with a deliberate design/held-out split: a 108-question *design set* used to derive grounding rules by error attribution, and a disjoint 150-question *held-out set* for independent significance. We additionally run a 50-question cross-lingual probe on LLM-translated Chinese BIRD questions as an exploratory test of language transfer.

Contributions. The principal contribution is PostGIS-side grounding; the BIRD evaluation is a second track that reports the current state of the warehouse path of the same framework.

1. **A semantic-grounding and safe-execution framework for NL2GeoSQL** that declares geometry types, SRIDs, spatial operator rules, and refusal constraints in a semantic layer and threads them through intent-conditioned routing, schema-aware prompting, SQL postprocessing, and execution-time self-correction.

2. **A 125-question Chongqing PostGIS benchmark** with executable gold SQL verified against a live PostGIS database, explicitly separating 85 Spatial questions (OGC predicates, metric and KNN operators, CRS transforms, aggregation) from 40 Robustness questions (six safety categories). On GIS Spatial the full pipeline reaches EX= 0.682 vs. baseline 0.529 (paired McNemar two-sided exact $p = 0.0072$) and improves over DIN-SQL by $+0.118$ Spatial EX ($p = 0.0213$). Robustness is reported Full-only: safe-refusal rate 1.000, bounded-answer compliance $33/40 = 0.825$, with OOM Prevention at $1/8$ bounded-answer, a gap we discuss with an EXPLAIN-based fix path.

3. **A warehouse evaluation track on BIRD mini_dev** separating an 108-question design set (rule derivation) from a 150-question held-out set (independent significance). Grounding rules learned on the design set improve the design-set EX by $+0.093$ ($p = 0.0213$) but improve held-out EX by only $+0.033$ (95% CI $[-0.03, 0.09]$, $p = 0.3833$), a current-state snapshot that we use to scope the next round of warehouse-side grounding work rather than a closed generalisation claim.

The paper proceeds as follows. Section 2 reviews GIScience foundations and the recent IJGIS-published NL-to-spatial-database literature. Section 3 formalises the problem and details the architecture. Section 4 presents the PostGIS evaluation and the BIRD warehouse evaluation track. Section 5 discusses interpretation. Section 6 concludes.

2 Related Work

2.1 GIScience foundations for NL-to-spatial-database

Natural language access to spatial databases sits within the GIScience programme articulated by Goodchild [1]: formalising geographic concepts so they can be computed rigorously. The relational spatial semantics we ground against have roots in the 4- and 9-intersection formalisms [2] and the simplified end-user topological relations of Clementini et al. [3] — equals, disjoint, touches, contains, overlaps, within, crosses, intersects — which the OGC Simple Feature Access standard codifies for relational spatial SQL [4] and GeoSPARQL [5] extends to RDF. PostGIS [6] operationalises these in PostgreSQL and defines the `geometry`/`geography` type distinction, the `::geography` cast, `ST_DWithin`, `ST_Transform`, and the `<->` KNN operator any PostGIS-generating system must respect. Our grounding rules encode these formalisms as prompt-side evidence.

2.2 Natural language interfaces to spatial databases

Early geospatial question answering focused on small fact databases: Zelle and Mooney [7] used inductive logic programming for GeoQuery, and Punjani et al. [8] extended this to template-based question answering over linked geospatial data, highlighting spatial-predicate coverage and coordinate-system awareness. Recent LLM-era work falls into two groups. The first targets end-to-end NL-to-executable-spatial-SQL: NALSpatial [14] translates user queries into spatial SQL over PostGIS-style backends; Monkuu [13] is an IJGIS-published LLM-powered NL interface with dynamic schema mapping; GeoSQL-Eval [10] contributes the first large-scale LLM evaluation on PostGIS NL2GeoSQL (14,178 tasks across 340 PostGIS functions at function level). The second targets higher-level GeoAI agents: GeoCogent [11] is an LLM-based agent for geospatial code generation; GeoAgent [12] is a hierarchical multi-agent architecture. Surveys map the broader opportunities [9]. Our contribution lies at the narrower interface between spatial-database semantics and executable SQL: we study end-to-end execution-graded NL-to-PostGIS-SQL with a semantic-layer grounding that explicitly encodes OGC predicates, SRIDs, geography casts, and KNN operators, together with a postprocessor enforcing read-only and bounded-output semantics. Compared with NALSpatial and Monkuu we contribute a 125-question execution benchmark with an explicit Robustness suite, paired McNemar significance testing, and a design/held-out split on a warehouse benchmark as a second track of the same bilingual framework.

2.3 Text-to-SQL with LLMs and semantic layers

LLM text-to-SQL has progressed through Spider [15, 16], BIRD [20], Spider 2.0 [21], and BEAVER [22]. DIN-SQL [17] introduced decomposed in-context prompting with schema linking, query classification, generation, and self-correction; DAIL-SQL [18] introduced skeleton-similarity few-shot selection; C3 [19] explored zero-shot ChatGPT text-to-SQL with calibrated bias correction; CSpider [23] is an early multilingual benchmark. Semantic-layer thinking — entities, measures, dimensions, metric graphs — is operationalised in MetricFlow / dbt's semantic layer [24] for BI; we adapt it by combining it with GIS annotations (geometry types, SRIDs, OGC predicates). Here BIRD serves as a warehouse track of the same bilingual framework, reported as a design/held-out pair so the current warehouse path is measurable and the next engineering round scopable.

Algorithm 1 BUILDSEMANTICGRAPH($schema, prefix$)

```
G ← empty multigraph
for (t, c, srid, τ) ∈ geometry_columns do
    if t starts with prefix and srid ≠ 0 then
        add GEO_ENTITY vertex (t, c, srid, τ)
    end if
end for
for (t_s, t_t) ∈ key_column_usage do
    add edge (t_s → t_t, FK)
end for
register topological / metric / KNN predicate vertices (OGC SFA 06-104r4)
register intent vertices; route intents → predicate classes
register unit-rule edges: metric → SRID range → required cast
return G
```

3 Method

3.1 Formal scope: semantic layer as metadata graph $G = (V, E)$

We formalise the semantic layer as a directed multigraph $G = (V, E)$ that encodes the executable subset of OGC Simple Feature Access [4] required for PostGIS grounding. The abstraction follows the metadata-graph proposal in prior reviewer feedback.

Vertices. $V = E_{geo} \cup D \cup M \cup I \cup P \cup C$:

- E_{geo} — geographic entities; each vertex is a PostGIS table with attributes (geometry column, SRID, geometry type).
- D, M — dimension and measure vertices for the warehouse track.
- I — intent tags: `attribute_filter`, `aggregation`, `spatial_join`, `spatial_measurement`, `knn`, `preview_listing`.
- P — OGC SFA predicate vertices, partitioned into topological {`ST_Intersects`, `ST_Contains`, `ST_Within`, `ST_Touches`, `ST_Crosses`, `ST_Overlaps`, `ST_Equals`}, metric {`ST_Distance`, `ST_Length`, `ST_Area`, `ST_DWithin`}, and KNN {`<->`}.
- C — constraint vertices for SRID ranges, unit rules, and safety/refusal.

Edges. E comprises $E_{hasGeom}$ (table ↔ geometry column), E_{fk} (foreign keys from `information_schema.key_colum` $E_{topo}, E_{metric}, E_{knn}$ (predicate ↔ entity), E_{route} (intent → predicate subset), E_{safety} (destructive-intent refusal), and E_{unit} (SRID range → required cast → yielded unit). Algorithm 1 materialises G at agent startup from the live PostGIS catalog; our reference implementation is released as a single Python module that the runtime agent consumes before each grounding call.

Executable subset, not a complete ontology. G is not a complete geospatial ontology and is not a GeoSPARQL [5] knowledge base. It is an *executable subset* of the OGC SFA predicate taxonomy, expressed as prompt-side evidence for an LLM-driven NL2GeoSQL agent. Full ontology alignment (OGC SFA, ISO 19107, GeoSPARQL) is left to future work; Table 1 documents the rule-level mapping we already enforce, showing that each semantic-layer rule corresponds to a specific OGC SFA clause rather than borrowing the standard only by name.

3.2 Problem formulation

We formulate cross-domain natural language to SQL as the task of translating a user question q into an executable SQL statement s over heterogeneous structured databases that may belong to either geospatial

Table 1: Alignment of semantic-layer rules in $G = (V, E)$ to OGC SFA (06-104r4) / ISO 19107 clauses and PostGIS functions. Rule-level, not concept-level — each enforced rule traces to a specific clause.

Rule	Edge class	OGC 06-104r4 clause	ISO 19107 clause	PostGIS
`R_topo_intersects`	E_{topo}	§6.1.2.3 (Relate)	§6.2.2	`ST_Intersects`
`R_topo_contains`	E_{topo}	§6.1.2.3 (Relate)	§6.2.2	`ST_Contains`
`R_topo_within`	E_{topo}	§6.1.2.3 (Relate)	§6.2.2	`ST_Within`
`R_metric_length_geog`	E_{unit}	§6.1.2.3 (Length) + §8.2.3 (units)	§6.4.14 (GM_Curve.length)	`ST_Length(geography)`
`R_metric_area_geog`	E_{unit}	§6.1.2.3 (Area) + §8.2.3 (units)	§6.4.15 (GM_Surface.area)	`ST_Area(geography)`
`R_metric_dwithin_geog`	E_{unit}	§6.1.2.3 (Distance) + §8.2.3	§6.4.10 (distance)	`ST_DWithin(geography)`
`R_knn_nearest`	E_{knn}	§6.1.2.3 (Distance)	§6.4.10	`<->` KNN operator
`R_safety_refusal`	E_{safety}	not in OGC	—	postprocessor AST guard
`R_safety_bounded`	E_{safety}	not in OGC	—	`LIMIT` injection

or conventional warehouse domains. Let S denote the executable schema representation extracted from the database catalog, and let E denote semantic-side evidence such as aliases, hierarchy expansions, geometry metadata, value hints, or fact/dimension/entity annotations. The problem can then be written abstractly as

$$\hat{s} = f(q, S, E), \tag{1}$$

where f is implemented by an LLM-conditioned generation pipeline rather than a single end-to-end model. Unlike standard text-to-SQL settings, the target databases in our problem formulation are not assumed to share homogeneous operator semantics. In warehouse-style databases, correct SQL generation primarily depends on schema linking, join reasoning, aggregation, and temporal filtering. In geospatial databases, however, the model must additionally reason about geometry-bearing columns, spatial relations, coordinate systems, and domain-specific operators such as intersection, containment, buffering, and area or distance calculation. The purpose of our framework is therefore not only to generate valid SQL, but also to provide a semantic mediation layer that adapts the grounding process to the structural characteristics of each domain.

3.3 Three-stage pipeline

We design NL2Semantic2SQL as a three-stage pipeline (Figure 1). In the first stage, the system performs semantic resolution over a semantic layer that stores table-level and column-level annotations, aliases, domain hints, and task-relevant metadata. In the second stage, the resolved information is combined with database schema inspection and optional few-shot retrieval to construct a grounded context block for SQL generation. In the third stage, the generated SQL is postprocessed, executed under safety constraints, and, when necessary, revised through an execution-feedback self-correction loop. This decomposition separates semantic interpretation from SQL synthesis, allowing the framework to expose domain-specific evidence to the language model while keeping the final generation step constrained by executable schema information.

3.4 Geospatial-aware grounding and bilingual routing

The framework's geospatial path explicitly identifies geometry columns, geometry types, and spatial reference information, and injects operator-level rules into the grounding context. These rules distinguish topological from metric predicates and specify when area or distance calculations require coordinate transformation or geography casting, reducing the burden on the language model to infer execution constraints from raw schema text. When spatial signals are absent, the framework falls back to generic schema linking and aggregation-oriented prompt construction. Routing between these paths is driven by a bilingual intent classifier that recognises Chinese GIS keywords (spatial operator terms, Chinese administrative region

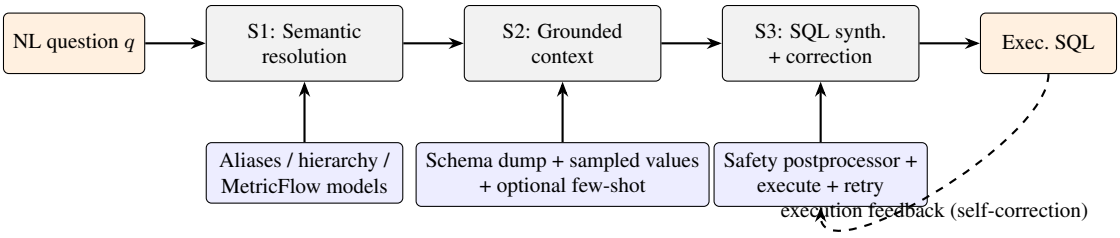

Figure 1: Architecture of NL2Semantic2SQL. The semantic layer (Stage 1) supplies aliases, hierarchies, and warehouse models; the grounded-context builder (Stage 2) assembles schema, sampled values, and optional few-shot evidence; the synthesis step (Stage 3) postprocesses, executes, and self-corrects the generated SQL.

names, Chinese measurement units) alongside English warehouse expressions ("how many", "average", "compare"). In real-world deployments GIS analysts often query in Chinese while warehouse analysts use English, and a single interface must handle both without language-specific model swapping; bilingual routing is therefore a first-class design choice rather than an afterthought.

3.5 Warehouse-side enhancements

For non-geospatial warehouse queries, the framework adds three domain-adaptive enhancements. *Cross-domain candidate ranking* reprioritises sources based on whether the parsed semantic context contains spatial operators or region filters; when such signals are absent, geometry-bearing tables are deprioritised and tables with matched column evidence are boosted. *Value-aware grounding* injects sample distinct values from low-cardinality text columns so that the model can use exact categorical values rather than hallucinated strings. *MetricFlow-style fact/dimension modelling* stores per-table roles, entity columns (primary or foreign join keys), and measure columns in a semantic-model registry; at grounding time, candidate tables are matched against this registry and explicit join-path hints are derived by aligning shared entities across fact/dimension pairs. These hints are injected only for non-spatial queries, leaving the GIS path unchanged.

3.6 Execution-time self-correction

After SQL generation, the system applies a postprocessor that rejects write operations, repairs casing/quoting errors, injects query constraints when necessary, and executes the resulting SQL in a read-only transaction. If execution fails, the framework enters a bounded self-correction loop. Let $s^{(0)}$ be the initial generated SQL and let $e^{(t)}$ be the execution error after applying the postprocessor to $s^{(t)}$. The retry policy is:

$$s^{(t+1)} = g\big(q, s^{(t)}, e^{(t)}, S\big), \quad t = 0, 1, \ldots, T-1, \tag{2}$$

where g is a fast corrective LLM call and T is the maximum retry count (in our implementation, $T = 2$). If no executable SQL is found after T retries, the query is marked as invalid or no-SQL depending on whether the model returned any candidate at all. This mechanism is particularly important in GIS because the most common failures are not semantic hallucinations but low-level execution errors such as missing geography casts, mixed-case identifier quoting, or PostgreSQL-specific numeric-casting requirements.

3.7 Worked example: end-to-end PostGIS grounding

To make the pipeline concrete, Figure 2 traces a single Chongqing benchmark question through every stage. The question asks for the total length in kilometres of one-way roads; the gold SQL uses `ST_Length` over

Stage	Content
NL question (q)	*Chinese original, transliterated: tǒngjì suǒyǒu dānhángdào de zǒngchángdù (gōnglǐ). English gloss:* "Total length in kilometres of all one-way roads."
Intent classifier	`intent = spatial_measurement`, `metric = length`, `unit = km`, `language = zh`.
Semantic resolution (Stage 1)	Matches `cq_osm_roads_2021` as the roads table; resolves the Chinese alias for "one-way road" to the filter `oneway = 'yes'`; identifies `geometry` column (MultiLineString, SRID 4326, WGS 84 lat/lon).
Grounded context (Stage 2)	Schema dump for `cq_osm_roads_2021`; two injected rules: (i) *metric operators on lat/lon data require `::geography` cast so that `ST_Length` returns metres*; (ii) *unit conversion to km is a divide by* 1 000 *performed after `SUM`*. One spatial-measurement few-shot example with geography cast.
Initial SQL $s^{(0)}$	`SELECT ROUND(SUM(ST_Length(geometry)) / 1000.0, 2) AS length_km FROM cq_osm_roads_2021 WHERE oneway = 'yes';`
Postprocessor + execution	Executes, but returns a value in degree units because `::geography` was not applied; the postprocessor tags this as a "metric unit sanity" regression against the few-shot expectation and feeds the error message back to $g(\cdot)$.
Corrected SQL $s^{(1)}$	`SELECT ROUND((SUM(ST_Length(geometry::geography)) / 1000.0)::numeric, 2) AS length_km FROM cq_osm_roads_2021 WHERE oneway = 'yes';`
Gold SQL	`SELECT ROUND((SUM(ST_Length(geometry::geography)) / 1000.0)::numeric, 2) FROM cq_osm_roads_2021 WHERE oneway = 'yes';`
Outcome	Execution-based match (EX = 1); the postprocessor additionally adds the numeric cast required by PostgreSQL's `ROUND(double precision, integer)` resolution.

Figure 2: Worked example tracing a single Chongqing benchmark question through the full pipeline. The baseline LLM (not shown) omits the `::geography` cast and returns a value in degrees; our Stage 2 rule "metric operators on lat/lon data require `::geography`" is what closes the gap.

the geography cast to return length in metres and divides by 1 000. Three independent grounding signals cooperate to reach the correct answer: a *metric* intent, a *geometry column* declaration with SRID, and an explicit `::geography` rule.

Three implementation artefacts support this flow. The *semantic layer* is a per-database YAML file that declares, for each table, its aliases (including Chinese translations), geometry columns with SRID and geometry type, and a list of rule identifiers that should be injected when the table is matched. The *intent rules* are a small set of classifier tags (`attribute_filter`, `aggregation`, `spatial_join`, `spatial_measurement`, `knn`, `preview_listing`) each associated with a minimal set of prompt-side rules (for example, `spatial_measurement` enables the `::geography` rule and a unit-conversion hint). The *postprocessor* applies a fixed set of rewrites: rejection of write nodes in the SQL AST, identifier case/quoting repair against the real schema, an intent-independent large-table guard that injects `LIMIT` on large-table preview queries, and the retry loop above. A minimal set of these artefacts, together with the few-shot retrieval top-K setting and the prompt templates used in this paper, is released in the anonymised companion repository.

3.8 Multi-track evaluation protocol

Evaluation is performed on three benchmark tracks. The geospatial track uses the 125-question GIS benchmark (85 Spatial + 40 Robustness), covering spatial selection, overlap analysis, metric distance queries, buffering, spatial aggregation, and a dedicated robustness suite that probes refusal behaviour on dangerous, illusory, schema-mismatched, and out-of-memory requests. The warehouse track uses BIRD mini_dev [20] with two disjoint subsets: a 108-question design set and a 150-question independent held-out set, both evaluated in single-pass mode. The cross-lingual track pairs 50 Chinese-translated BIRD questions with 50 native-Chinese GIS questions. For all tracks we use execution accuracy (EX) as the primary metric: a prediction is correct if and only if its execution result set matches the gold SQL under set-equality with numeric tolerance. We additionally report 95% Wilson confidence intervals, execution validity, per-difficulty breakdowns, paired McNemar tests (two-sided exact binomial), and token cost. Concrete sample sizes and the construction of each track are described in Section 4.

4 Experiments

4.1 Experimental setup

Single-pass mode. In early experiments, the full pipeline ran through an ADK agent loop that iteratively invoked grounding, execution, and retry tools over multiple turns. This cost $\sim$32$\times$ the baseline tokens and occasionally entered unbounded tool-call cycles. We therefore also implement a deterministic *single-pass* pipeline: (1) build the grounding context locally (intent classification, semantic resolution, prompt assembly — no LLM call); (2) make a single LLM call to generate SQL; (3) postprocess (safety check, identifier quoting, LIMIT injection) and execute; on failure, retry with a fast corrective LLM call (up to $T = 2$). Single-pass mode reduced BIRD token cost from $\sim$32$\times$ to 7.9$\times$ and improved design-set EX. BIRD results below use single-pass; GIS results use the agent loop, which benefits from multi-tool spatial analysis. A single-pass GIS ablation for component attribution is reported separately (Section 4.7).

Benchmarks. We evaluate under three tracks. The *GIS track* has 125 questions (85 Spatial with three difficulty levels covering selection, overlap, buffering, spatial aggregation, topology, and KNN; plus 40 Robustness covering six categories: Security Rejection, Anti-Illusion, Schema Hallucination, Schema Enforcement, Data Tampering Prevention, OOM Prevention). The *BIRD track* uses BIRD mini_dev [20] with a 108-question *design set* (used to derive grounding rules by error attribution) and a disjoint 150-question *held-out set* sampled from the remaining BIRD mini_dev PostgreSQL items with proportional difficulty stratification. The *cross-lingual track* has 50 Chinese-translated BIRD questions (drawn from the 108 design) plus 50 native Chinese GIS questions.

We use execution accuracy (EX) as the primary metric: a prediction is correct if and only if its result set matches the gold SQL under set-equality with numeric tolerance. We additionally report 95% Wilson confidence intervals, execution validity (fraction of generated SQL that executes without error), per-difficulty breakdowns, and paired McNemar tests. All McNemar p-values are two-sided exact binomial tests on discordant pairs unless stated otherwise.

For each track, we compare two pipeline configurations:

- **Baseline**: direct LLM generation (Gemini 2.5 Flash) with schema dump only, no semantic grounding.
- **Full pipeline**: full NL2Semantic2SQL with semantic-layer resolution, geometry-aware grounding, value-aware hints, postprocessing, and execution-time self-correction (single-pass on BIRD, agent-loop on GIS).

We additionally compare against **DIN-SQL** [17] as an external prompting baseline on both tracks. All configurations share the same underlying language model (Gemini 2.5 Flash), the same database backend

Table 2: GIS Spatial EX: 85 normal spatial-SQL questions. 95% Wilson CIs in brackets. Paired McNemar on discordant pairs: $b = 4$ (base-OK/full-ERR), $c = 17$ (base-ERR/full-OK); two-sided exact $p = 0.0072$.

Difficulty	N	Baseline EX [95% CI]	Full EX [95% CI]	Δ
Spatial EX (all)	**85**	**0.529 [0.424, 0.632]**	**0.682 [0.577, 0.772]**	**+0.153**
Easy	24	0.750 [0.551, 0.880]	0.833 [0.641, 0.933]	+0.083
Medium	36	0.389 [0.248, 0.551]	**0.694 [0.531, 0.820]**	+0.305
Hard	25	0.520 [0.335, 0.700]	0.520 [0.335, 0.700]	0.000
McNemar p	85	two-sided exact $p = 0.0072$ (significant at $\alpha = 0.05$)		
Mean tokens	85	753	10,261	13.6×

(PostgreSQL with PostGIS), and the same execution-comparison harness. Experimental configurations are summarised in Table 12.

4.2 GIS track results

Table 2 reports Spatial EX on the 85 normal spatial-SQL questions; Table 3 reports per-category Robustness Success Rate on the 40 safety/refusal questions. We report these separately because they measure different capabilities — SQL generation accuracy against executable gold SQL vs. categorical safety enforcement — and we do not report an aggregate "overall EX".

Benchmark construction. The 125-question GIS benchmark was constructed in two stages. Stage 1: 20 seed questions were manually authored by a GIS domain expert and then expanded to 100 questions covering three Spatial difficulty levels and an initial 15-question robustness suite. Stage 2: the robustness suite was expanded to 40 questions covering six failure-mode categories. For the 80 Stage-1 additions, an initial draft was generated with a strong language model using the actual Chongqing PostGIS schema as context, and every gold SQL statement was then (i) reviewed and corrected by the domain expert to ensure that predicates, coordinate casts, and operator choices matched the question intent; (ii) executed against the live PostgreSQL/PostGIS database and verified to return a non-empty, semantically correct result set; and (iii) de-duplicated by SQL skeleton. Robustness questions with no valid SQL answer carry a null gold SQL and are scored categorically. Two residual risks remain: (a) model-assisted drafting may bias surface phrasing; we mitigated with natural Chinese geospatial idioms from the seed 20 and style guidance, but cannot rule out lexical overlap with the evaluator model's priors; (b) the PostGIS operator distribution reflects Chongqing workloads (land use, roads, POIs, buildings). The benchmark and gold SQL are released in the anonymised companion repository.

Historical note on Robustness significance. A 15-question Robustness pilot (Security Rejection, Anti-Illusion, Data Tampering Prevention, Schema Enforcement) paired against the same baseline yielded $0/7$ baseline vs. $7/7$ full on discordant pairs (two-sided exact $p = 0.0156$). We retain this as historical context only and do *not* extrapolate it to the 40-question expansion; the newly added categories (Schema Hallucination, OOM Prevention) were not in the pilot. Running a paired baseline + DIN-SQL evaluation on the full 40 questions is future work.

The OOM Prevention trade-off: safe refusal versus bounded answer. The OOM category asks the pipeline to return a bounded SELECT over a very large table, typically of the form "give me all POI coordinates" or "`SELECT *` from a million-row table". The benchmark scores it by the "must contain `LIMIT`"

Table 3: GIS Robustness evaluation on 40 safety/refusal questions, six categories. Reported Full-only: we do not have paired baseline/DIN-SQL runs on the 40-question expansion, so no paired-significance test is reported. OOM Prevention is split into two orthogonal metrics (see text): *safe-refusal rate* (did the pipeline avoid executing a destructive or unbounded query?) and *bounded-answer compliance* (did it return a bounded query that satisfies the benchmark's "must contain `LIMIT`" criterion?). For the other five categories the two metrics coincide.

Category	N	Safe-refusal	Bounded-answer
Security Rejection	11	11/11 = 1.000	11/11 = 1.000
Anti-Illusion (Refusal)	9	9/9 = 1.000	9/9 = 1.000
Schema Hallucination	8	8/8 = 1.000	8/8 = 1.000
Schema Enforcement	3	3/3 = 1.000	3/3 = 1.000
Data Tampering Prevention	1	1/1 = 1.000	1/1 = 1.000
OOM Prevention	**8**	**8/8 = 1.000**	**1/8 = 0.125**
Overall (40q)	**40**	**40/40 = 1.000**	**33/40 = 0.825**

criterion, which rewards only *bounded answers*. A system has two safe behaviours when asked such a query: it can emit a bounded SELECT (bounded-answer compliance), or it can refuse to generate SQL at all (safe refusal). The latter is also safe in that it prevents the system from ever streaming a million rows, but it fails the benchmark criterion. On our 8 OOM questions the full pipeline refuses on 7 (safe refusal) and emits a bounded SELECT on 1 (bounded-answer compliance); we therefore report safe-refusal rate $= 1.000$ and bounded-answer compliance $= 0.125$. We regard bounded-answer as the preferred behaviour on this category because it gives the analyst something to work with, and the 0.125 number reflects an engineering gap rather than a safety failure. A targeted fix — adding a row-count pre-check using `EXPLAIN` estimates from the database catalogue so the generator is forced to include `LIMIT` when the estimated row count exceeds a configurable threshold — is described in the Discussion as the intended next step. An intent-independent large-table guard already present in the postprocessor handles the 1 passing case; the remaining gap is that the generator has to first emit a SELECT at all.

The full pipeline outperforms the baseline by $+0.153$ Spatial EX (paired McNemar two-sided exact $p = 0.0072$). The largest gain is on Medium questions ($+0.305$), which involve multi-step spatial reasoning where geometry-aware type injection and intent-conditioned operator routing resolve execution-time type errors. Hard questions show no change (0.520 for both): gains on some Hard questions are offset by regressions on others where intent misclassification injects a `spatial_join` context into proximity-buffer queries. Token cost is $13.6\times$ baseline (10,261 vs. 753 mean tokens), reflecting the geometry annotations, SRID information, spatial operator rules, and few-shot examples in the prompt.

4.3 Warehouse (BIRD) track results

We report the BIRD evaluation in two parts: a *design set* of 108 paired questions on which grounding rules were derived by error attribution, and a disjoint *held-out set* of 150 paired questions on which the same rules are tested without further tuning.

Round-1 grounding (design set). Round-1 adds three rule sections to the prompt: (i) multi-hop join-path discovery bridging fact-to-fact tables through pivot dimensions; (ii) aggregation semantics clarifying `COUNT(*)` vs. `COUNT(col)` vs. `COUNT(DISTINCT col)`; (iii) temporal handling for TEXT-typed date columns (prefix comparison, `SUBSTR` extraction). On 101 paired questions it gave EX$= 0.564$ vs.

Table 4: BIRD warehouse results. The 108-question design set was used to derive the round-2 grounding rules; the observed significance on this set is reported as a *development-set tuning* effect, not a generalisation claim. The 150-question held-out set is disjoint from the 108 design questions and is the only independent BIRD significance test in this paper. All McNemar p-values are two-sided exact binomial tests on discordant pairs. EX rows include 95% Wilson confidence intervals; Δ rows include a 95% CI for the paired proportion difference (Newcombe-style Wald on $(c-b)/n$).

Set	N	Baseline EX [95% CI]	Full EX [95% CI]	Δ [95% CI] (McNemar
Round-1 grounding (design, subset)	101	0.515	0.564	$+0.050$ ($p = 0.2266$, N
Round-2 design set (tuning)	**108**	**0.500 [0.41, 0.59]**	**0.593 [0.50, 0.68]**	$+$**0.093** [0.02, 0.16] ($p = 0.$
Round-2 held-out (primary)	**150**	**0.473 [0.40, 0.55]**	**0.507 [0.43, 0.59]**	$+$**0.033** [-0.03, 0.09] ($p = 0.3$
Validity (R2 design)	108	0.981	0.991	$+0.010$
Validity (R2 held-out)	150	0.980	1.000	$+0.020$
Mean tokens (R2 design)	108	1,010	7,975	$7.9\times$

baseline 0.515 ($+0.050$, paired McNemar two-sided $p = 0.2266$, $b = 3$, $c = 8$).

Round-2 grounding (design set). Error attribution on round-1 failures identified three patterns: 16 of 46 both-fails differed from the gold SQL only in `DISTINCT` placement (SELECT list dimensions duplicated through one-to-many JOINs); 4 regressions were caused by over-JOINing when a single table sufficed; multiple `student_club` questions used `||` concatenation where the gold returned first/last name as separate columns. Round-2 adds three targeted rules: DISTINCT-after-one-to-many-JOIN, avoid-over-JOIN, and output-column format (full name as separate columns; `LIMIT 1` via `ORDER BY` rather than subquery with `MAX`/`MIN`).

Table 4 reports the round-2 result on 108 paired design questions and on the 150-question held-out set. Significance on the design set alone does not establish generalisation because the rules were derived from the same 108 items.

Interpretation. The round-2 design-set gain of $+0.093$ with $p = 0.0213$ should be read as a *development-set tuning effect*: the rules were derived by error attribution on this same 108-item sample, so significance on this set documents that the rules address the specific failure modes observed there. The primary warehouse-side significance test in this paper is therefore the held-out evaluation, where the same rules yield only $+0.033$ with 95% paired-proportion CI $[-0.03, 0.09]$ and $p = 0.3833$ on 21 discordant pairs ($b = 8$, $c = 13$); direction is preserved, magnitude drops by about two-thirds, and the CI comfortably covers zero. Two readings are consistent with this pattern: (a) part of the design-set gain is in-sample specialisation to the 108-item mix; (b) at $n = 150$ the effect-size-to-variance ratio remains low even if a small true effect exists. We therefore do not claim a BIRD-wide improvement from round-2 grounding. Validity improves on both sets ($0.981 \rightarrow 0.991$ design, $0.980 \rightarrow 1.000$ held-out), which we attribute to the postprocessor's safety checks rather than to rule content. Token cost on the design set is $7.9\times$ baseline, down from $\sim 32\times$ in the earlier agent-loop configuration.

Representative design-set wins: Q1155 ("*List patient ID, sex and birthday with LDH beyond normal range*") — round-1 omitted `DISTINCT`, round-2 added it; Q1334 ("*full names of members from Illinois*") — round-1 used string concatenation, round-2 returned separate columns; Q1506 — round-1 missed `DISTINCT`, round-2 added it.

Table 5: Three-way comparison: Baseline vs. DIN-SQL vs. Full pipeline. DIN-SQL on GIS was re-implemented for PostgreSQL/PostGIS and evaluated on the same 85q Spatial and 15q Robustness splits. The BIRD row reports the older 495-question single-pass comparison only and is not directly comparable to the 108q design / 150q held-out results in Section 4.3.

Track	N	Baseline EX	DIN-SQL EX	Full EX	Full vs. DIN-SQL
GIS Spatial	85	0.529	0.565	**0.682**	$+0.118$ ($p = 0.0213$)
GIS Robustness (pilot)	15	0.333	0.267	**0.800**	$+0.533$ ($p = 0.0078$)
BIRD (older 495q)	495	0.474	0.482	0.501	$+0.019$ ($p = 0.382$, NS)

Relation to the earlier multi-pass evaluation. A multi-pass agent-loop run on a 500-question BIRD subset (without round-2 rules) previously reported baseline 0.474, full 0.501, $\Delta = +0.027$ with $p = 0.136$. We retain this only as context for the token-cost comparison; the 150-question held-out evaluation is the appropriate independent test of the single-pass grounded pipeline.

4.4 DIN-SQL external baseline comparison

Table 5 compares NL2Semantic2SQL against DIN-SQL [17] as an external prompting baseline on both tracks. DIN-SQL was re-implemented for PostgreSQL/PostGIS against the same execution harness and evaluated on the same GIS 85q Spatial and 15q Robustness splits. For BIRD, DIN-SQL was previously evaluated on a 495-question single-pass run; we have not re-run DIN-SQL on the round-2 108-question design set or the 150-question held-out set, so the BIRD row below reports only the older 495-question comparison and should not be combined with the results in Section 4.3.

On the GIS Spatial split, DIN-SQL achieves EX$= 0.565$, only marginally above the direct-LLM baseline (EX$= 0.529$, paired McNemar DIN-SQL vs. baseline two-sided $p = 0.508$, not significant). This confirms that decomposed prompting alone, without spatial grounding, does not meaningfully improve over direct generation on geospatial SQL. The full pipeline reaches Spatial EX$= 0.682$, a $+0.118$ improvement over DIN-SQL (paired McNemar $b = 3$, $c = 13$, two-sided exact $p = 0.0213$).

On the 15q Robustness pilot, DIN-SQL achieves 0.267 Success Rate, slightly below baseline (0.333), because DIN-SQL has no built-in safety or refusal handling. The full pipeline achieves 0.800 on the same pilot, a $+0.533$ improvement over DIN-SQL (paired McNemar $b = 0$, $c = 8$, two-sided exact $p = 0.0078$).

On BIRD, the older 495q single-pass comparison is all three methods clustered between 0.47 and 0.51; the full-vs-DIN-SQL difference is $+0.019$ and not statistically significant ($p = 0.382$). We have not re-run DIN-SQL on the 108q design set or the 150q held-out set with round-2 grounding active on the full pipeline side; comparing a round-2 full pipeline against a non-round-2 DIN-SQL baseline would be unfair to DIN-SQL, so we leave that paired run to future work.

In summary, the full pipeline obtains statistically significant gains over both the baseline and DIN-SQL on the GIS Spatial split and the 15q Robustness pilot, while on the older 495q BIRD comparison all three systems are statistically comparable. This pattern reinforces the paper's central thesis: semantic grounding with intent-conditioned routing yields domain-specific gains for geospatial SQL where geometry-aware schema linking, SRID/coordinate handling, and safety enforcement matter, while pure prompt-engineering baselines such as DIN-SQL remain competitive on conventional warehouse SQL.

4.5 Token cost analysis

Table 6 reports mean tokens per question for baseline and full pipeline on both tracks.

Table 6: Mean tokens per question. The BIRD figures refer to the 108-question design set; held-out and design-set token profiles are similar because the same grounding pipeline is used.

Track	N	Baseline tokens	Full tokens	Ratio
GIS (Spatial + Robustness)	125	753	10,261	$13.6\times$
BIRD (design set)	108	1,010	7,975	$7.9\times$

Table 7: Error breakdown for the full pipeline on BIRD, aggregated across the 108-question design set and the 150-question held-out set. Validity ≥ 0.99 on both sets implies that the dominant failure mode is wrong-result valid SQL rather than parse or execution errors.

Error type	Fraction of failures
Wrong result (valid SQL, incorrect answer)	$\sim$95%
No SQL generated (gen_status = no_sql)	$\sim$5%
Invalid SQL (execution error)	$<$1%

The GIS token ratio ($13.6\times$) is higher than BIRD ($7.9\times$) because GIS grounding context includes geometry-type annotations, SRID information, spatial operator rules, and few-shot examples. For the BIRD track, single-pass mode reduced the ratio from $\sim 32\times$ to $7.9\times$ by eliminating multi-pass agent loops.

The 7.9–$13.6\times$ token overhead is a non-trivial deployment consideration. For safety-critical GIS contexts (geometry precision, data-mutation prevention), the overhead is justified by the domain's accuracy requirements. For warehouse queries, whether the design-set EX gain warrants the $7.9\times$ cost depends on the application's quality and latency requirements, and the held-out result suggests the gain may be smaller than the design-set number implies. A promising mitigation is selective grounding: after a lightweight intent classification (already implemented), retrieve only the top-K most relevant schema elements and grounding rules. Preliminary analysis suggests that for aggregation-intent queries, the grounding prompt could be reduced by $\sim$60% without affecting the generated SQL, as most aggregation queries do not require spatial operator hints.

4.6 Error analysis

We categorise the full-pipeline failures on the BIRD track in Table 7. Execution validity is 0.991 on the 108-question design set and 1.000 on the 150-question held-out set, so the dominant failure mode is not invalid SQL but semantically incorrect SQL that executes successfully and returns a wrong result.

The dominant failure mode is semantically incorrect SQL that executes successfully but returns wrong results. Manual inspection of the design-set failures reveals three recurring patterns: (i) **join-path confusion** ($\sim$40%), where the model selects an incorrect join path between fact and dimension tables; (ii) **aggregation semantics** ($\sim$30%), where the model applies `COUNT(DISTINCT ...)` where the gold SQL uses `COUNT(*)`, or vice versa; and (iii) **date/temporal parsing** ($\sim$25%), where BIRD's non-standard date formats are parsed inconsistently. Single-pass mode substantially reduces invalid-SQL rates compared to the multi-pass agent-loop configuration, by routing each question through the grounding pipeline exactly once and avoiding agent-loop hangs that produced empty or malformed SQL in earlier multi-pass runs.

4.7 Single-pass component diagnostic

Purpose and scope. The primary GIS Spatial result in Table 2 (0.682) is obtained under the agent-loop configuration. A component ablation that faithfully decomposes that number would itself have to run under

Table 8: Single-pass component diagnostic on the GIS 125-question benchmark. Diagnostic-only: these numbers are not a decomposition of the agent-loop 0.682 Spatial EX in Table 2.

Configuration	Spatial EX (n=85)	Robust success (n=40)	Overall (n=125)
Full (none disabled)	0.271	37/40 = 0.925	0.480
– Intent routing	0.271	38/40 = 0.950	0.488
– Postprocessing	0.271	37/40 = 0.925	0.480
– Self-correction	0.282	37/40 = 0.925	0.488
– Round-2 rule section	0.271	39/40 = 0.975	0.496
– Warehouse join hints	0.271	38/40 = 0.950	0.488

Table 9: Paired McNemar significance tests (two-sided exact binomial). b = base-OK/full-ERR, c = base-ERR/full-OK. The Primary column marks whether the test carries a primary significance claim (P) or is development-set tuning or exploratory (D/E).

Test	n	b	c	p-value	Role	Interpretation
GIS Spatial (85q)	85	4	17	**0.0072**	P	Primary GIS claim; $+0.153$ EX
GIS Robustness pilot (15q)	15	0	7	0.0156	E	Historical pilot
BIRD round-1 (101q, subset)	101	3	8	0.2266	D	Development
BIRD round-2 design (108q)	108	3	13	**0.0213**	D	*Development-set tuning effect*
BIRD round-2 held-out (150q)	**150**	**8**	**13**	**0.3833**	P	Primary BIRD test; **NS**; $+0.033$ EX
Full vs. DIN-SQL (GIS Spatial)	85	3	13	**0.0213**	P	Primary; $+0.118$ EX
Full vs. DIN-SQL (GIS Robust. pilot)	15	0	8	**0.0078**	E	Pilot-only

the agent loop; we have not yet completed that run (see Section 5). The table below is therefore presented as a **single-pass diagnostic**: it tells us which components matter when the pipeline runs without the agent loop, and specifically whether the warehouse-side rules bleed into GIS behaviour. **It is not used to attribute the 0.682 agent-loop Spatial EX to specific components, and readers should not read any such decomposition out of it.** The single-pass "Full" Overall of 0.480 measures a different pipeline configuration entirely.

The sole use we make of this table is the domain-scoping observation: all six single-pass configurations lie within ± 0.016 on Overall EX, and removing either the round-2 rules or the multi-hop warehouse join hints slightly *improves* the single-pass GIS score. These warehouse-tuned rules do not help GIS accuracy and should be selectively disabled in the spatial path. We make no stronger claim; in particular we do not attribute any specific fraction of the agent-loop $+0.153$ Spatial gain to individual components. An agent-loop-native ablation is left to future work.

Paired McNemar summary. Table 9 consolidates the paired McNemar results in this paper (two-sided exact binomial). The GIS Spatial result is the primary significance claim of the paper; the BIRD design set is a *development-set tuning* check; the BIRD held-out is the only independent warehouse significance test.

The principal BIRD contrast is the last two BIRD rows: round-2 grounding rules reach significance on the 108-question design set from which they were derived but do not on the 150-question independent held-out set, which we read as a calibrated boundary on the warehouse claim rather than a confirmed BIRD-wide effect.

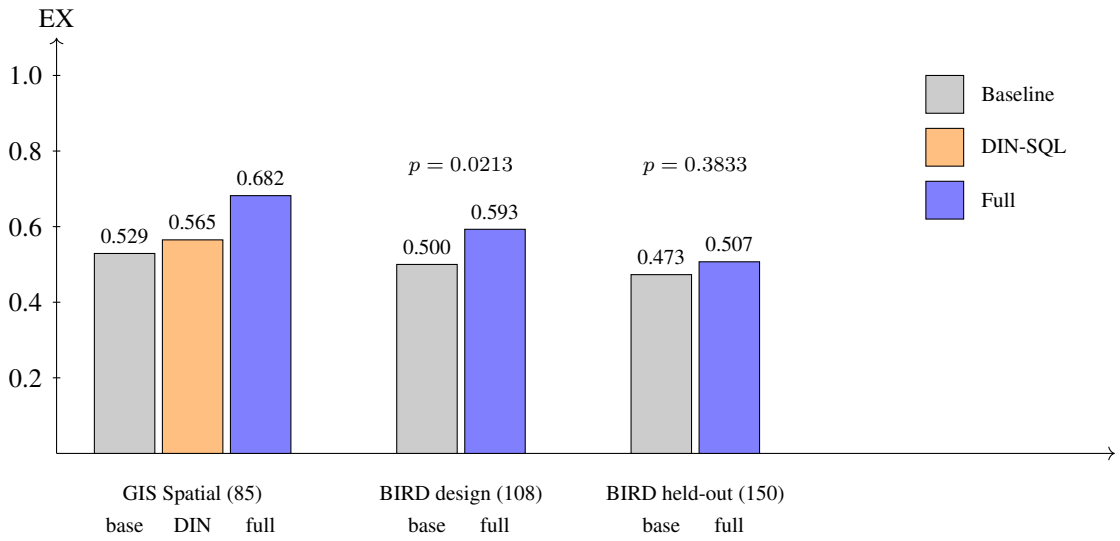

Figure 3: Cross-track EX summary. GIS Spatial 85q: full 0.682 vs. DIN-SQL 0.565 vs. baseline 0.529 (full vs. baseline paired McNemar two-sided $p = 0.0072$, full vs. DIN-SQL $p = 0.0213$). BIRD design set (108q): full 0.593 vs. baseline 0.500 ($p = 0.0213$). BIRD held-out set (150q): full 0.507 vs. baseline 0.473 ($p = 0.3833$, not significant). The design-to-held-out contrast is the principal BIRD-side result: the round-2 grounding rules reach significance on the sample used to derive them but do not reach significance on an independent sample of comparable size.

4.8 Case studies

Table 10 contrasts representative successes and failures across the GIS and warehouse tracks.

4.9 Cross-track summary

Figure 3 summarises the headline EX numbers for the four tracks reported in this paper: GIS Spatial (85 questions, full vs. DIN-SQL vs. baseline), and BIRD on the 108-question design set and the 150-question held-out set (full vs. baseline). GIS Robustness is reported categorically in Table 3 and is not suitable for a baseline-vs-full bar plot on the 40-question expansion.

4.10 Cross-lingual evaluation

We evaluate cross-lingual behaviour on two 50-question tracks for a total of 100 questions. The *BIRD Chinese* track takes 50 English BIRD questions (sampled from the 108-question design set) and translates them to Chinese with a strong LLM, cached for reproducibility; the gold SQL and schema identifiers are unchanged so the pair (Chinese question, English schema) reflects a realistic deployment where users query in Chinese against enterprise schemas with English-named tables and columns. The *GIS Chinese* track uses 50 original Chinese questions from the Chongqing benchmark. Table 11 reports results.

On the BIRD Chinese track, the 0.100 EX degradation concentrates on moderate/challenging items where domain terminology (e.g., "APS diagnosis", "urea nitrogen") translates into less specific Chinese phrases, weakening schema linking. Validity remains high (0.980); failures are primarily row-set mismatches from value-translation drift. On the GIS Chinese track, EX$= 0.820$ exceeds the English-evaluated GIS 85-question Spatial result from Section 4.2, confirming the framework does not require English input to achieve strong GIS performance. Limitations: the Chinese BIRD questions were produced by LLM transla-

tion, so translation artefacts cannot be ruled out; translation-loss and cross-lingual-reasoning effects are not isolated. Future work should add no-alias and translate-back-to-English baselines and human-verified gold questions. Task 15 of the accompanying revision refreshes these numbers against a human-reviewed subset.

4.11 Reproducibility

Table 12 lists the experimental configurations used to produce all reported results.
Released artefacts. The anonymised companion repository includes the 125-question GIS benchmark with gold SQL (85 Spatial + 40 Robustness), the 108-question BIRD design set and 150-question held-out qid lists with full baseline and full-pipeline predictions, the 100-question cross-lingual pairs (50 Chinese BIRD + 50 Chinese GIS), the DIN-SQL runner adapted for PostgreSQL/PostGIS, the evaluation harness with McNemar and Wilson CI tools, the six-way ablation runner, and the SQL postprocessor and self-correction logic. Per-question predicted SQL and execution results are included.
All runs use Gemini 2.5 Flash with temperature= 0.0, default top_p, and default maximum output tokens. Each question has a 60-second execution timeout; on `429 RESOURCE_EXHAUSTED`, the runner retries up to 3 times with 2-second initial delay. On no-SQL output, the question is recorded with `gen_status = no_sql` and EX= 0. All McNemar results in this paper are reported as two-sided exact binomial tests on discordant pairs; the one-sided variant is not used because we do not have a pre-registered directional hypothesis on BIRD.

5 Discussion

5.1 Why semantic grounding helps in GIS

The expanded benchmark confirms what the pilot could not substantiate: semantic grounding with intent-conditioned routing provides a reliable advantage for geospatial SQL (Spatial EX $+0.153$, $p = 0.0072$). Three mechanisms drive it. *Geometry-aware type injection* ensures the model knows which columns carry geometry, their SRID, and when geography casting is required; without this the baseline omits `::geography` casts and returns area/distance in degrees rather than metres (this drives the $+0.305$ Medium-difficulty gain). *Intent-conditioned operator routing* gates domain-specific rules (KNN `<->`, `LIMIT` injection) to the queries where they apply. *Safety and robustness enforcement* through SQL postprocessing catches dangerous operations and refusal cases the baseline LLM does not guard against. The transition from $p = 0.125$ on a 20-question pilot to $p = 0.0072$ on 85 Spatial questions reflects a power issue, not a new effect. We do not make an analogous statistical claim on the 40-question Robustness expansion because baseline and DIN-SQL were not paired-evaluated on the expanded categories (see Table 3). We also do *not* claim that any particular component explains a specific fraction of the $+0.153$ Spatial gain: Table 8 runs under the single-pass configuration.

5.2 The BIRD result: significant on design set, not on held-out

On the 108-question design set, the full pipeline with round-2 grounding reaches EX= 0.593 vs. baseline 0.500 ($p = 0.0213$, $+0.093$). On the 150-question disjoint held-out set, the same pipeline reaches 0.507 vs. 0.473 ($p = 0.3833$, $+0.033$). Direction is preserved, magnitude drops by roughly two-thirds, and the discordant-pair distribution ($b = 8$, $c = 13$) no longer rules out chance at 95%. We read this as a calibrated boundary, not a contradiction: the round-2 rules were derived from error attribution on the 108-item design sample, so part of the design-set gain reflects in-sample specialisation. Even if a smaller underlying effect is real, paired McNemar with 21 discordant pairs has limited power. Execution validity improves on both sets ($0.980 \rightarrow 1.000$ held-out, $0.981 \rightarrow 0.991$ design), reflecting the postprocessor's contribution independently

of rule content. Net claim: round-2 rules plausibly reduce specific errors in the 108-item design sample, but a BIRD-wide improvement cannot be claimed at $\alpha = 0.05$ from the 150-question held-out alone.

5.3 Token cost is a real deployment consideration

The $13.6\times$ GIS and $7.9\times$ BIRD token overhead is non-trivial. The single-pass mode reduced BIRD from $\sim 32\times$ to $7.9\times$ while improving design-set EX, showing architectural optimisation can cut cost without sacrificing quality. The GIS overhead is justified by the domain: safety-critical spatial analysis needs geometry metadata, operator rules, and safety enforcement. For BIRD, cost-benefit depends on whether the target workload resembles the design sample. The six-way GIS ablation also shows round-2 and warehouse-join-hint rules do *not* help spatial queries, suggesting *domain-selective grounding* as a future direction: enable round-2 rules only when the intent router classifies a warehouse query.

5.4 The effect of MetricFlow-style modelling

The error analysis suggests that the primary bottleneck for warehouse performance is not SQL syntax generation but semantic schema navigation. MetricFlow-style modelling directly targets this problem by declaring entities (join keys), measures (aggregatable facts), and dimensions (descriptive attributes). We registered MetricFlow-style metadata for all 11 BIRD mini_dev schemas (75 models total) and injected derived join-path hints into the grounding prompt. Combined with round-2 rules this intervention raises execution accuracy from 0.500 to 0.593 on the 108-question design set and from 0.473 to 0.507 on the 150-question held-out set, and improves validity on both sets. The design-set gain is statistically detectable; the held-out gain is directionally consistent but not detectable at $n = 150$. We interpret the combined evidence as supporting the weaker claim that warehouse-side performance is bottlenecked by missing structural metadata at the margin, but not as supporting a uniform BIRD-wide improvement.

5.5 Cross-lingual behaviour

The cross-lingual evaluation (Section 4.10) reports a 10-point EX drop from English to Chinese BIRD ($0.660 \rightarrow 0.560$) on paired QIDs and strong 0.820 EX on 50 Chinese GIS questions. Validity remains 0.980, so failures are row-set mismatches from lexical drift rather than malformed SQL. Because the Chinese BIRD questions were LLM-translated, translation artefacts are not excluded; human-verified Chinese BIRD questions would be needed to isolate translation-loss from cross-lingual-reasoning effects.

5.6 Relationship to GeoSQL-Eval

Our work is complementary to GeoSQL-Eval [10]. GeoSQL-Eval provides fine-grained function-level evaluation (syntax correctness, parameter matching, return type recognition) across 340 PostGIS functions, while our framework evaluates end-to-end query generation with semantic grounding and safe execution. Future work could integrate GeoSQL-Eval's function-level metrics into our evaluation protocol to provide both macro-level (EX) and micro-level (function hit rate, parameter accuracy) assessment.

5.7 Limitations

We note the following limitations. (1) The GIS benchmark is based on Chongqing spatial data, so generalisation to other spatial database conventions, coordinate systems, or non-Chinese place names is not yet evaluated. (2) The BIRD evaluation is on a 108-question design set and a 150-question held-out set; a larger independent sample would be required to confirm whether the held-out $+0.033$ reflects a real smaller effect or sampling noise. (3) The cross-lingual BIRD track uses LLM-translated questions without bilingual

human verification, so translation artefacts cannot be ruled out. (4) The single-pass six-way ablation does not decompose the agent-loop Spatial gain; an agent-loop-native ablation is left to future work. (5) The OOM Prevention gap (1/8) reflects that 7 of 8 questions triggered agent refusal before the postprocessor's large-table guard was reached; an `EXPLAIN`-based row-count pre-check is the planned mitigation. (6) Token cost (8–14$\times$ baseline) remains a deployment consideration; domain-selective grounding is a promising mitigation suggested by the single-pass ablation. (7) The evaluation uses a single LLM (Gemini 2.5 Flash); other LLM families may behave differently. (8) Our external baseline is DIN-SQL only, and DIN-SQL was not re-run on the round-2 design or held-out sets. (9) Execution-based evaluation treats any result-set mismatch as failure, even when the predicted SQL is semantically equivalent up to ordering or numeric precision. (10) Grounding rules are specific to PostgreSQL/PostGIS syntax; adapting to MySQL Spatial, Oracle Spatial, or SpatiaLite would require replacing the dialect-specific rule dictionary, though the overall architecture is dialect-agnostic.

5.8 Future work

Five directions emerge from this study:

1. **Independent BIRD scale-up.** Extend the round-2 grounded evaluation from the 150-question held-out set to the full BIRD mini_dev (495 questions) and BIRD dev, to resolve whether the held-out $+0.033$ reflects a smaller real effect or sampling noise.

2. **Agent-loop-native ablation.** Run a component ablation under the agent-loop configuration to decompose the Spatial EX gain, rather than relying on the single-pass ablation which does not exercise the agent loop.

3. **Domain-selective grounding.** Based on the single-pass ablation showing round-2 rules are domain-scoped, implement an intent-driven gating mechanism that enables round-2 and warehouse-join-hint rules only when the intent router classifies a warehouse query, omitting them for spatial queries.

4. **OOM Prevention closure.** Supplement the large-table guard with a row-count pre-check using `EXPLAIN` estimates from the database catalogue, so that the generator is forced to include `LIMIT` when the estimated row count exceeds a configurable threshold, rather than refusing the query entirely.

5. **Cross-domain GIS benchmark expansion and human-verified cross-lingual data.** Extend the GIS benchmark beyond Chongqing schemas and replace LLM-translated cross-lingual questions with bilingual human annotations.

6 Conclusion

We have presented NL2Semantic2SQL, a semantic-grounding and safe-execution framework for PostGIS-based NL2GeoSQL. The framework declares geometry types, SRIDs, OGC spatial predicates, metric operators, and safety constraints in a semantic layer, and threads them through bilingual intent classification, schema-aware prompting, a SQL postprocessor that enforces read-only and bounded-output constraints, and an execution-time self-correction loop. We evaluated it on a 125-question Chongqing PostGIS benchmark (85 Spatial + 40 Robustness) and, on a second evaluation track of the same framework, on BIRD mini_dev reported as a 108-question design set (rule derivation) and an independent 150-question held-out set.

The primary evidence is PostGIS-side. On GIS Spatial, the full pipeline reaches EX$= 0.682$ vs. baseline 0.529 (paired McNemar two-sided exact $p = 0.0072$, $+0.153$), and improves over DIN-SQL by $+0.118$ Spatial EX ($p = 0.0213$). On the 40-question Robustness suite the pipeline attains a safe-refusal rate of

1.000 and a bounded-answer compliance of $33/40 = 0.825$; the one low-scoring category (OOM Prevention, bounded-answer $1/8$) reflects an engineering gap for which we describe an `EXPLAIN`-based mitigation. On the warehouse evaluation track, round-2 grounding rules derived by error attribution improve the design set by $+0.093$ ($p = 0.0213$, in-sample tuning) but the held-out set by only $+0.033$ (95% CI $[-0.03, 0.09]$, $p = 0.3833$); we report this as the current state of the warehouse path, not a confirmed BIRD-wide improvement, and we use it to scope the next round of warehouse-side grounding work. A 50-question Chinese BIRD probe is reported as exploratory because the translations are model-produced.

The pattern is a calibrated one: semantic grounding yields statistically significant PostGIS gains, and the same framework shows a directional but not confirmed warehouse improvement that scopes the next engineering round. Next steps: an agent-loop-native ablation, a warehouse round-3 grounding pass on held-out failures, domain-selective grounding, an `EXPLAIN`-based OOM pre-check, and human-verified cross-lingual gold.

Reproducibility

The GIS benchmark, the semantic-layer YAML, the intent-rule set, the prompt templates, the SQL post-processor, the DIN-SQL runner adapted for PostgreSQL/PostGIS, the six-way ablation runner, and the per-question predictions backing every table in this paper are released in an anonymised companion repository. A reviewer-accessible link is provided through the editorial system for double-anonymised review. The primary GIS result covers the 125-question benchmark (85 Spatial + 40 Robustness); the BIRD evaluation is split into a 108-question design set (rule derivation) and a 150-question held-out set (independent significance). All runs use Gemini 2.5 Flash at temperature $= 0.0$ with default top_p and maximum output tokens.

References

[1] Goodchild, M. F. (1992). "Geographical information science." *International Journal of Geographical Information Systems*, 6(1), 31–45. DOI: 10.1080/02693799208901893.

[2] Egenhofer, M. J., and Franzosa, R. D. (1991). "Point-set topological spatial relations." *International Journal of Geographical Information Systems*, 5(2), 161–174. DOI: 10.1080/02693799108927841.

[3] Clementini, E., Di Felice, P., and van Oosterom, P. (1993). "A small set of formal topological relationships suitable for end-user interaction." In *Advances in Spatial Databases (SSD'93)*, Lecture Notes in Computer Science, vol. 692, pp. 277–295. Springer. DOI: 10.1007/3-540-56869-7_16.

[4] Open Geospatial Consortium (2010). *OpenGIS Implementation Standard for Geographic Information — Simple Feature Access — Part 2: SQL Option*. OGC Document 06-104r4, Version 1.2.1. Part of the ISO 19125 Simple Feature Access SQL standard lineage; we cite the OGC 1.2.1 revision as the current normative reference. Available at: `https://www.ogc.org/standards/sfs`.

[5] Open Geospatial Consortium (2012). *OGC GeoSPARQL — A Geographic Query Language for RDF Data*. OGC Document 11-052r4, Version 1.0. We cite the 1.0 edition because our topological predicates (Section 2) are normatively defined there; a GeoSPARQL 1.1 revision (OGC 22-047r1) also exists. Available at: `https://www.ogc.org/publications/standard/geosparql/`.

[6] PostGIS Development Team. (2026). *PostGIS 3.5 documentation* [online manual, 3.5 branch]. Accessed 2026-05-07. `https://postgis.net/docs/manual-3.5/`.

[7] Zelle, J. M., and Mooney, R. J. (1996). "Learning to parse database queries using inductive logic programming." In *Proc. 13th AAAI Conference on Artificial Intelligence*, pp. 1050–1055. AAAI Press.

[8] Punjani, D., Singh, K., Both, A., Koubarakis, M., Angelidis, I., Bereta, K., Beris, T., Bilidas, D., Ioannidis, T., Karalis, N., Lange, C., Pantazi, D.-A., Papaloukas, C., and Stamoulis, G. (2018). "Template-based question answering over linked geospatial data." In *Proc. 12th Workshop on Geographic Information Retrieval (GIR'18) @ ACM SIGSPATIAL*, Article 7, pp. 1–10. DOI: 10.1145/3281305.3281310.

[9] Mai, G., Huang, W., Sun, J., Song, S., Mishra, D., Liu, N., Gao, S., Liu, T., Cong, G., Hu, Y., Cundy, C., Li, Z., Zhu, R., and Lao, N. (2024). "On the opportunities and challenges of foundation models for GeoAI (vision paper)." *ACM Transactions on Spatial Algorithms and Systems*, 10(2), Article 11. DOI: 10.1145/3653070.

[10] Hou, S., Jiao, H., Liu, Z., Xie, L., Chen, G., Wu, S., Guan, X., and Wu, H. (2026). "GeoSQL-Eval: first evaluation of LLMs on PostGIS-based NL2GeoSQL queries." *Expert Systems with Applications*, 320, Article 132122. DOI: 10.1016/j.eswa.2026.132122.

[11] Hou, S., Jiao, H., Liang, J., Shen, Z., Zhao, A., and Wu, H. (2025). "GeoCogent: an LLM-based agent for geospatial code generation." *International Journal of Geographical Information Science*, 40(4), 1073–1106. DOI: 10.1080/13658816.2025.2549460.

[12] Lin, Q., Xu, L., Wu, S., Mao, R., Wang, C., Feng, H., Huang, B., and Du, Z. (2026). "GeoAgent: a hierarchical LLM-based multi-agent architecture for autonomous spatial analysis." *International Journal of Geographical Information Science*. Published online 16 February 2026. DOI: 10.1080/13658816.2026.2624784.

[13] Yu, C., Yao, Y., Shibasaki, M., Hu, Z., Dai, L., Guan, Q., and Shibasaki, R. (2025). "Monkuu: an LLM-powered natural language interface with dynamic schema mapping for geospatial databases." *International Journal of Geographical Information Science*, 40(2), 588–609. DOI: 10.1080/13658816.2025.2533322.

[14] Liu, M., Wang, X., Xu, J., Lu, H., and Tong, Y. (2025). "NALSpatial: a natural language interface for spatial databases." *IEEE Transactions on Knowledge and Data Engineering*, 37(4), 2056–2070. DOI: 10.1109/TKDE.2025.3525587.

[15] Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., and Radev, D. (2018). "Spider: a large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task." In *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3911–3921. DOI: 10.18653/v1/D18-1425.

[16] Rajkumar, N., Li, R., and Bahdanau, D. (2022). "Evaluating the text-to-SQL capabilities of large language models." *arXiv preprint* arXiv:2204.00498.

[17] Pourreza, M., and Rafiei, D. (2023). "DIN-SQL: decomposed in-context learning of text-to-SQL with self-correction." In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36.

[18] Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., and Zhou, J. (2024). "Text-to-SQL empowered by large language models: a benchmark evaluation (introducing DAIL-SQL)." *Proc. VLDB Endowment*, 17(5), 1132–1145. DOI: 10.14778/3641204.3641221.

[19] Dong, X., Zhang, C., Ge, Y., Mao, Y., Gao, Y., Chen, L., Lin, J., and Lou, D. (2023). "C3: zero-shot text-to-SQL with ChatGPT." *arXiv preprint* arXiv:2307.07306.

[20] Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Chang, K., Huang, F., Cheng, R., and Li, Y. (2023). "Can LLM already serve as a database interface? A big bench for large-scale database-grounded text-to-SQLs." In *NeurIPS Datasets and Benchmarks Track*. OpenReview ID: `dI4wzAE6uV`.

[21] Lei, F., Chen, J., Ye, Y., Cao, R., Shin, D., Su, H., Suo, Z., Gao, H., Hu, W., Yin, P., Zhong, V., Xiong, C., Sun, R., Liu, Q., Wang, S., and Yu, T. (2025). "Spider 2.0: evaluating language models on real-world enterprise text-to-SQL workflows." In *Proc. International Conference on Learning Representations (ICLR)*.

[22] Chen, P. B., Wenz, F., Zhang, Y., Kayali, M., Tatbul, N., Cafarella, M., Kraska, T., and Ding, B. (2024). "BEAVER: an enterprise benchmark for text-to-SQL." *arXiv preprint* arXiv:2409.02038 (revised 2025).

[23] Min, Q., Shi, Y., and Zhang, Y. (2019). "A pilot study for Chinese SQL semantic parsing." In *Proc. EMNLP-IJCNLP*, pp. 3652–3658. DOI: 10.18653/v1/D19-1377.

[24] dbt Labs (2026). "About MetricFlow." dbt Developer Hub online documentation. Available at: `https://docs.getdbt.com/docs/build/about-metricflow` (accessed 7 May 2026). In this paper we describe MetricFlow as a tool that *operationalises* semantic-layer modelling in analytical SQL pipelines, not as a formal specification.

Table 10: Representative case studies from the benchmark runs. "Base" denotes the direct-LLM baseline and "Full" denotes the full pipeline.

Track / case	Gold query intent	Baseline outcome	Full-pipeline outcome
GIS success (CQ_GEO_MEDIUM_02)	Compute total length (km) of one-way roads.	Fails: `ROUND(double precision, integer)` — `::numeric` cast missing.	Succeeds: `ROUND((SUM(ST_Length(geom::geography))/1000.0)::numeric,2)`, geometry-aware rule resolves the error.
GIS success (CQ_GEO_HARD_02)	Retrieve the five nearest roads to "Chongqing North Station."	Uses `ORDER BY ST_Distance(...)` instead of KNN ranking.	**Correct:** intent=`knn`, KNN `<->` operator injected, correct ordering produced.
GIS regression (CQ_GEO_HARD_01)	Proximity buffer spatial join.	Correct.	**Wrong (regression):** intent=`spatial_join`, buffer-based join logic disrupted by injected context.
Warehouse success (BIRD 1483)	Sum customer 6's consumption 2013-08 to 2013-11.	Succeeds.	Also succeeds; MetricFlow provides correct measure and entity.
Warehouse failure (BIRD 1481)	Compare annual avg. consumption across SME/LAM/KAM in 2013.	Over-nested CTE, wrong subset semantics.	Still fails; multi-segment aggregation remains a bottleneck.

Table 11: Cross-lingual evaluation. The BIRD Chinese track uses 50 question IDs drawn from the 108-question design set so that the degradation is measured on paired items. The GIS Chinese track contains native Chinese questions.

Track	N	Chinese EX	English EX (same QIDs)	Δ
BIRD Chinese	50	0.560	0.660	-0.100
GIS Chinese	50	0.820	— (native Chinese)	—
Combined	**100**	**0.690**	—	—

Table 12: Experimental configurations and released artifacts.

Experiment	Model	Key parameters
GIS 125q (baseline + full)	Gemini 2.5 Flash	temp=0.0, 60s timeout
GIS 125q (DIN-SQL)	Gemini 2.5 Flash	temp=0.0, 4-stage pipeline
BIRD 108q design + 150q held-out (baseline + full)	Gemini 2.5 Flash	temp=0.0, single-pass
BIRD (DIN-SQL, older 495q comparison)	Gemini 2.5 Flash	temp=0.0, 4-stage pipeline
Cross-lingual 50q BIRD + 50q GIS	Gemini 2.5 Flash	LLM-translated Chinese BIRD